3 Deferred execution, IEnumerable, IQueryable və loading

Bu dərsdə yalnız dörd mövzunu eyni sifariş nümunəsi üzərində öyrənəcəyik:

- **Deferred execution:** sorğunu yazmaqla onu işlədib nəticə almağın fərqi.
- **IQueryable:** EF sorğusuna şərtlərin necə əlavə edilməsi.
- **IEnumerable:** elementləri oxumaq və C#-da filtrləmə; IQueryable ilə fərqi.
- **Loading:** eager, explicit və lazy ilə əlaqəli məhsulların nə vaxt gətirilməsi.

Loading hissəsində explicit və lazy-ni ayrıca müqayisə edəcəyik. Sonra N+1-in hər iki üsulla necə yarandığını və bütün məhsullar əvvəlcədən istənəndə sorğu sayının necə dəyişdiyini görəcəyik.

Əvvəl sorğunun özünü, sonra onun nəticəsindəki obyektin əlaqələrini araşdırırıq. Hər kodda iki suala cavab axtar: **SQL hansı sətirdə işləyir? Həmin sətirdən sonra hansı məlumat artıq proqramdadır?**

Nümunələr real SQLite bazasında, EF Core 10.0.9 ilə yoxlanılıb. Hər təcrübə eyni məlumatı olan təzə DbContext ilə başlayır. SQL saylarına başlanğıc məlumatını yaradan əmrlər daxil deyil.

Nümunələrdə `sql.Count` faktiki işlənmiş SQL əmrlərinin sayını göstərir. Kod parçalarını təkrarlamaq üçün işlək layihə mövcuddur. Həmin qovluqda `dotnet run -- query`, `dotnet run -- enumerable`, `dotnet run -- eager`, `dotnet run -- explicit` və `dotnet run -- lazy` əmrləri uyğun nümunəni işlədir.

Nümunədəki məlumatı tanıyaq

Bazada Orders sifarişləri, OrderItems isə onların məhsul sətirlərini saxlayır. Order 42 Ayselə aiddir; iki məhsulu var: Cay və Tort. Order 43 Murada aiddir, bir məhsulu var. Order 44 Leylaya aiddir, məhsulu yoxdur.

EF Core baza məlumatı ilə C# obyektləri arasında işləyən kitabxanadır. **SQL** bazaya sorğu göndərmək üçün dildir. **DbContext** həmin əməliyyatları idarə edən obyektdir; kodda adı `db`-dir. `db.Orders` Order-lar üçün sorğu yazmağa çıxış verir; onların artıq gətirildiyini bildirmir.

C# - dərsdə istifadə olunan obyektin quruluşu

```
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public virtual ICollection<OrderItem> Items { get; set; }
        = new List<OrderItem>();
}
```

Entity burada bir baza qeydini təmsil edən C# obyektidir. Order sifarişi, OrderItem isə bir məhsul sətrini təmsil edir. OrderItem-in Id, OrderId, ProductName xassələri var. OrderId onun hansı sifarişə aid olduğunu göstərən nömrədir.

Navigation property əlaqəli obyektlərə keçid verən xassədir: Items sifarişin məhsul obyektlərinə keçiddir. `new List<OrderItem>()` sadəcə boş siyahı yaradır; bazadan məhsul oxumur. `virtual` sözünün lazy loading üçün rolunu sonra görəcəyik.

LINQ C#-da Where kimi metodlarla sorğu yazmaq imkanıdır. `Where(o => o.Id == 42)` “yalnız Id-si 42 olanları seç” deməkdir. `o` yoxlanılan sifarişi bildirən dəyişəndir; `==` bərabərlik yoxlamasıdır.

Sorğunu yazmaq hələ onu icra etmək deyil

Query - sorğu hansı məlumatı istədiyimizin təsviridir. **Execution - icra** həmin sorğunun işlədilib nəticəsinin alınmasıdır. Aşağıdakı iki sətir fərqli iş görür:

C# - bu iki sətri ayrı-ayrı oxu

```
var query = db.Orders.Where(o => o.Id == 42);

var orders = await query.ToListAsync();
```

Birinci sətir: “Orders-dən yalnız Id-si 42 olanları istəyəcəyəm” qaydasını qurur. Bu mərhələdə **SQL sayı 0-dır**.

İkinci sətir: “İndi həmin qaydanın nəticəsini gətir və siyahı hazırla” deyir. Bu nümunədə **bir SQL** icra olunur. `orders` artıq içində Order obyekti olan List-dir.

SQL - yaranan əmrin sadələşdirilmiş mənası

```
SELECT Id, CustomerName
FROM Orders
WHERE Id = 42;
```

`SELECT` gətiriləcək sütunları, `FROM` cədvəli, `WHERE` şərti bildirir. Provider-in faktiki əmri dırnaqlar və əlavə hissələr baxımından fərqlənə bilər.

Materialization gələn baza nəticəsindən C# obyektlərinin hazırlanmasıdır: EF nəticədəki 42 və Aysel dəyərlərini Order obyektinin xassələrinə yerləşdirir.

`ToListAsync` nəticəni siyahıya yığır. Async asinxron əməliyyatı bildirir; `await` nəticəni gözləyib sonra davam etməyə imkan verir. Async ayrıca loading növü deyil.

Deferred execution nə deməkdir?

Deferred execution - təxirə salınmış icra sorğunu qurarkən nəticənin hələ oxunmaması, oxuma işinin nəticəyə ehtiyac yaranan ana saxlanmasıdır.

C# - dotnet run -- foreach

```
var query = db.Orders.Where(o => o.Id == 42);
Console.WriteLine("Evvel SQL: " + sql.Count);

foreach (var order in query)
    Console.WriteLine("Id: " + order.Id);

Console.WriteLine("Sonra SQL: " + sql.Count);
```

Yoxlanmış çıxış

```
Evvel SQL: 0
Id: 42
Sonra SQL: 1
```

`sql` işlək layihədə icra olunmuş SQL əmrlərinin qeydlərini saxlayan siyahıdır. `sql.Count` həmin qeydlərin sayıdır.

`foreach` elementləri bir-bir oxuyan dövrdür. Sorğunun elementlərini oxumaq başlayanda bazaya müraciət edilir. Deməli, **SQL yalnız ToList ilə işləmir; foreach də EF sorğusunu icra edə bilər.**

`Where` və `OrderBy` kimi addımlar EF sorğusunu hazırlayır. `ToListAsync`, `FirstAsync`, `SingleAsync`, `CountAsync`, `AnyAsync` kimi nəticə istəyən əməliyyatlar onu icra edir.

Sorğu obyektinin mövcud olması nəticənin artıq gətirildiyini göstərmir. Sonrakı səhifələrdə məhz bu fərqi qoruyacağıq.

IQueryable nədir və bizə nə verir?

`IQueryable<Order>` Order sorğusu ilə işləmək üçün **interface**, yəni tipin təmin etdiyi imkanları bildirən müqavilədir. Buradakı `<Order>` sorğunun hansı tip elementlər üzərində qurulduğunu göstərir.

EF sorğusunu bu tipdə saxlayanda sonrakı uyğun Where kimi addımlar da bazaya göndəriləcək sorğunun təsvirinə əlavə oluna bilir:

C# - dotnet run -- query

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);

query = query.Where(o => o.Id == 43);
var rows = await query.ToListAsync();
```

Birinci şərt 43 və 44-ü uyğun sayır. İkinci şərt bunlardan yalnız 43-ü saxlayır. İcra anında hər iki şərt SQL-ə daxil olur. **Bazadan bir Order sətri gəlir: Muradın 43 nömrəli sifarişi.**

Provider EF sorğusunu konkret baza üçün SQL-ə çevirən hissədir. EF bu şərtlərin strukturunu saxlayır. Buna **expression tree - ifadə ağacı** deyilir: “Id sütununu 43-lə müqayisə et” kimi əməliyyatların proqram daxilində təsviri. Provider bu təsviri oxuyub uyğun SQL hazırlayır.

IQueryable özü “SQL bazası” demək deyil. Onun davranışını arxasındakı provider müəyyən edir. Bizim nümunədə provider EF Core-un SQLite provider-idir; buna görə uyğun şərtlər SQL-ə çevrilir.

İki Where yazdıq, amma iki SQL işləmədi. Sorğunu iki addımda qurub bir dəfə icra etdik.

IEnumerable nədir? Hazır siyahı deməkdirmi?

`IEnumerable<Order>` “bu mənbənin Order elementlərini bir-bir oxuya bilərsən” müqaviləsidir. Elementləri oxumağa **enumeration** deyilir. Adi foreach bu imkandan istifadə edir.

Bu müqavilə elementlərin haradan gələcəyini demir. Hazır List-dən də gələ bilər, hələ icra edilməmiş EF sorğusundan da. Buna görə **IEnumerable yazılması məlumatın artıq yaddaşda olması demək deyil.**

C# - burada hələ SQL sayı 0-dır

```
IEnumerable<Order> source = db.Orders;
```

Yalnız dəyişənin görünən tipi dəyişdi. Əsl mənbə hələ EF sorğusudur. `foreach` və ya `ToList()` ilə oxunanda bazaya müraciət edilə bilər.

IQueryable<T> həm də IEnumerable<T>-dir. Yəni sorğu qurmaq imkanı olan mənbənin elementlərini oxumaq imkanı da var. Əks istiqamət ümumi qayda deyil: hər IEnumerable SQL qura bilmir.

C# - hazır List nümunəsi

```
List<int> numbers = new() { 1, 2, 3 };
IEnumerable<int> source = numbers;
```

Bu ikinci nümunədə elementlər artıq List-dədir. Heç bir baza yoxdur. IEnumerable hər iki vəziyyətə uyğun gəldiyindən yalnız tipin adına baxıb “SQL işlənib” nəticəsini çıxara bilmərik.

List<T> elementləri saxlayan konkret siyahı tipidir. **IEnumerable<T>** isə onları oxumaq imkanını bildirir. Bu fərq sonrakı Where-in harada işləyəcəyinə təsir edir.

Eyni Where niyə fərqli yerdə işləyir?

İki nümunənin də yekun nəticəsi yalnız 43-dür. Amma bazadan gələn məlumatın miqdarı fərqlənir. Hər variant ayrıca təzə context-də işlədilir.

C# - IQueryable: hər iki şərt SQL-ə gedir

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);
var rows = await query.Where(o => o.Id == 43)
    .ToListAsync();
```

C# - IEnumerable: ikinci şərt C#-da işləyir

```
IEnumerable<Order> source = db.Orders
    .Where(o => o.Id >= 43);
var rows = source.Where(o => o.Id == 43).ToList();
```

Birincidə SQL həm `Id >= 43`, həm `Id == 43` şərtini daşıyır. Baza 43-ü qaytarır. **Bir Order obyekti alınır.**

İkincidə ilk Where tip dəyişməzdən əvvəl yazılıb və SQL-də qalır. Sonrakı Where IEnumerable üçün seçilir: gələn obyektlər C# kodu ilə yoxlanır. **Baza 43 və 44-ü qaytarır; C# 44-ü kənarlaşdırır.**

Ölçü	IQueryable variantı	IEnumerable variantı
İcra olunmuş SQL	1	1
Oxunmuş Order obyektləri	1	2
Son siyahıdakı Id	43	43

Eyni metod adı müxtəlif müqavilələr üçün təklif olunur. C# hansı variantı seçəcəyinə dəyişənin kodda görünən tipinə görə qərar verir. Burada ikinci variantın Where-i SQL təsviri qurmur; obyekt üzərində işləyən C# yoxlaması olur.

AsEnumerable və ToList fərqi

`AsEnumerable()` mənbəni IEnumerable kimi davam etdirməyə imkan verir. **Özü SQL işlətmir, özü List yaratmır.** Ondan sonrakı adi LINQ filtrləri gələn obyektlər üzərində C#-da işləyir:

C# - boundary rejimi

```
var query = db.Orders.Where(o => o.Id >= 43)
    .AsEnumerable()
    .Where(o => o.Id == 43);
// SQL: 0

var rows = query.ToList();
// SQL: 1; baza 43 və 44-ü gətirib, C# 43-ü saxlayıb.
```

AsEnumerable-dan əvvəlki Where SQL-də qalır. Sonrakı Where isə SQL-ə əlavə edilmir. ToList mənbəni oxuyub yekun elementləri hazır siyahıda saxlayır.

C# - əvvəl tam siyahını alsaq

```
var list = await db.Orders.ToListAsync();
var filtered = list.Where(o => o.Id == 43).ToList();
```

Bu ikinci nümunədə ilk sətirdə üç sifariş də artıq gəlib. İkinci sətir hazır obyektləri süzür və əlavə Orders SQL-i işləmir.

Server-side filtrin bazada hesablanmasıdır. **Client-side** filtrin EF-dən istifadə edən C# proqramında hesablanmasıdır; burada client sözü brauzer demək deyil.

AsEnumerable oxuma qaydasını dəyişir; ToList isə nəticəni həqiqətən oxuyub saxlayır. Bazadan az məlumat gətirmək istəyirsənsə uyğun filtrləri bu keçidlərdən əvvəl yaz.

Deferred yalnız IQueryable-a aid deyil

Hazır List üzərindəki Where də yoxlamanı nəticə oxunana qədər saxlaya bilər. Bu dəfə heç baza yoxdur:

C# - dotnet run -- list-deferred

```
var numbers = new List<int> { 1, 2 };
IEnumerable<int> query = numbers.Where(n => n > 1);

numbers.Add(3);
Console.WriteLine(string.Join(",", query));
```

Yoxlanmış nəticə

```
2,3
```

Where yazılan anda siyahıda 1 və 2 var idi. Lakin yoxlama hələ yerinə yetirilməmişdi. Sonra 3 əlavə olundu. `string.Join` nəticəni mətnə çevirmək üçün elementləri oxuyanda Where artıq 3-ü də gördü.

Deməli, **IEnumerable həmişə immediate, IQueryable həmişə deferred** kimi əzbərləmək yanlışdır. **Immediate execution** əməliyyat çağırışı zamanı nəticənin hesablanmasıdır; deferred isə nəticənin oxunmasını sonraya saxlayır.

Hazır siyahının özü artıq mövcud məlumatdır. Onun üzərində qurulan yeni Where nəticəsi isə hələ hesablanmamış ola bilər. Mənbənin hazır olması ilə yeni sorğunun icra olunması ayrı hadisələrdir.

Əgər `var result = numbers.Where(n => n > 1).ToList();` yazıb yalnız bundan sonra 3 əlavə etsəydik, result-da yalnız 2 qalardı. Çünki siyahı əvvəlki anda hazırlanmışdı.

İndi loading sualına keçirik

İndiyədək sorğunun **nə vaxt** icra olunduğunu və filtrin **harada** işlədiyini öyrəndik. İndi başqa sual var: **Order gələndə onun Items məlumatı da gəlirmi?**

Bu dərsdə loading əlaqəli məlumatın gətirilməsidir. Üç üsul həmin məlumatı hansı addımda istədiyimizi fərqli şəkildə ifadə edir:

Üsul	Məhsulların gətirilməsi necə tələb olunur?
Eager loading	Order sorğusuna əvvəlcədən Include əlavə edirik.
Explicit loading	Order gələndən sonra ayrıca LoadAsync əmri yazırıq.
Lazy loading	Mexanizm aktivdirsə yüklənməmiş Items oxunuşu yükləməni başladır.

Eager “bu kod sətrini yazan kimi SQL işləyir” demək deyil. Include ilə qurulmuş sorğu da ToListAsync və ya SingleAsync kimi icra addımını gözləyə bilər.

C# - hələ SQL yoxdur

```
var query = db.Orders.Include(o => o.Items);

// İcra burada başlayır.
var order = await query.SingleAsync(o => o.Id == 42);
```

Deferred execution əsas sorğunun icra anıdır. Lazy loading isə alınmış obyektin əlaqəsini sonradan gətirmək mexanizmidir. Hər ikisində gecikmə ola bilər, amma gecikdirilən iş eyni deyil.

Sonrakı nümunələrdə lazy yalnız lazy sınağında aktivdir. Avtomatik Include ayarı yoxdur və hər sınaq təzə context ilə başlayır.

Heç loading istəməsək nə gəlir?

C# - yalnız Order gətiririk

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.CustomerName);
Console.WriteLine(order.Items.Count);
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
```

Yoxlanmış nəticə - lazy bağlıdır

```
Aysel
0
False
SQL: 1
```

Baza Orders-dən 42-ni qaytardı, EF Order obyektini hazırladı. Bu sorğuda OrderItems istənməyib. Sinifdəki `new List<OrderItem>()` boş siyahı yaratdığı üçün Items.Count 0 görünür. **Bazadakı iki məhsul itməyib; sadəcə bu obyektin Items-inə gətirilməyib.**

`Count` burada mövcud kolleksiyadakı element sayıdır. `db.Entry(order)` EF-in həmin obyektlə bağlı idarəetmə məlumatına çıxış verir. `Collection(o => o.Items)` Items əlaqəsini seçir. `IsLoaded` EF-in onu yüklənmiş sayıb-saymadığını bildirir.

False yüklənmiş sayılmadığını göstərir. True və False məntiqi doğru və yanlış dəyərlərdir.

Bu təcrübədə context təzədir və məhsullar əvvəl alınmayıb. Əsas fakt budur: **Order obyekti var, amma Items bazadan oxunmayıb.**

Eager: məhsulların lazım olduğunu əvvəlcədən de

Eager loading əlaqəli məlumatı əsas sorğu ilə birlikdə tələb etməkdir. Burada bunu Include ilə ifadə edirik:

C# - eager sınağı

```
var query = db.Orders.Include(o => o.Items);
// SQL: 0

var order = await query.SingleAsync(o => o.Id == 42);
// SQL: 1, Order və onun 2 məhsulu artıq gəlib.

Console.WriteLine(order.Items.Count);
// 2; SQL yenə 1-dir.
```

`Include(o => o.Items)` “alacağın sifarişlərin Items əlaqəsini də doldur” tələbidir. Sorğunu yalnız qurur; icranı SingleAsync başladır.

SingleAsync-dən sonrakı vəziyyət belədir:

C# obyektlərinin məntiqi görünüşü

```
order: Id = 42, CustomerName = Aysel
  Items[0]: Cay
  Items[1]: Tort
```

Yəni Items.Count oxunanda məhsulları gətirmək üçün əlavə SQL lazım deyil. Onlar artıq gəlib.

Bu nümunədə bir SQL işləyir. Eager-in əsas mənası **əlaqəni əvvəlcədən sorğuda istəməkdir**. Hər layihədə mütləq bir SQL olacağını düşünmək əvəzinə faktiki əmrlərin sayını yoxla.

Əgər ekranda sifarişlə məhsullar mütləq göstəriləcəksə, bu tələbi sorğunun özündə görmək rahatdır.

Explicit: məhsulları ayrıca əmrlə istə

Explicit loading əlaqəni yükləmək üçün açıq şəkildə ayrıca əmr yazmaqdır. Əvvəl Order gəlir, sonra lazım olsa Items gətirilir:

C# - explicit sınağı

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; Items hələ gətirilməyib.

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
// SQL: 2; indi Items-də 2 məhsul var.

Console.WriteLine(order.Items.Count); // 2
// SQL yenə 2-dir.
```

LoadAsync “bu konkret order obyektinin Items əlaqəsini yüklə” deyir. EF order.Id-nin 42 olduğunu bildiyi üçün məhsul sorğusuna bu şərti qoyur:

SQL - ikinci əmrin sadələşdirilmiş mənası

```
SELECT Id, OrderId, ProductName
FROM OrderItems
WHERE OrderId = 42;
```

Burada əlavə SQL-i Items.Count başlamadı. **Əlavə SQL-i LoadAsync sətri başladı.** Count oxunanda artıq gəlmiş siyahının sayı alınır.

Məsələn, sifarişi gətirəndən sonra iş qaydasına əsasən məhsulların lazım olub-olmadığını qərarlaşdırırsansa bu qərarın altına LoadAsync yaza bilərsən. Əlaqə artıq yüklənmiş sayılırsa eyni LoadAsync çağırışını təkrarlamaq bu sınaqda yeni SQL yaratmır; bu, məcburi yeniləmə əmri deyil.

Lazy: əvvəl sifariş gəlir, məhsullar gözləyir

Lazy loading artıq alınmış entity-nin hələ yüklənməmiş əlaqəsini, həmin əlaqəyə müraciət ediləndə gətirməkdir. Sifarişin özünü gətirməyi gözlətmirik; bu nümunədə gözləyən iş onun məhsullarını gətirməkdir.

Bazada Order 42 Ayselə aiddir və iki məhsulu var: Cay və Tort. Lazy aktiv, context təzədir və əlaqəni əvvəlcədən istəmirik:

C# - birinci SQL burada işləyir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
```

Bu sətirdən sonra Order-un Id və CustomerName məlumatları gəlib. **Items-in yüklənməsi isə hələ baş verməyib.**

C# - bu oxunuş ikinci SQL yaratmır

```
Console.WriteLine(order.CustomerName); // Aysel
```

CustomerName sifarişin öz məlumatıdır; birinci nəticə ilə artıq gətirilib. Items isə başqa cədvəldəki əlaqəli məhsullara keçiddir:

C# - ikinci SQL məhz burada işləyir

```
var items = order.Items;
```

Bu sətir “42-nin məhsullarını gətir” işini başladır. Məhsullar gəlib kolleksiyaya yerləşəndən sonra items dəyişəni həmin kolleksiyanı alır.

“Lazım olanda gətirir” ifadəsində proqramın fikrini təxmin etmək yoxdur. Tətik konkret əməliyyatdır: yüklənməmiş Items xassəsini oxumaq. Items heç oxunmasa, bu təcrübədə məhsul SQL-i də işləməz.

Adi xassə oxunuşu SQL-i necə işlədir?

Özümüz yazdığımız adi List avtomatik bazaya qoşulmur. Bunun üçün burada **lazy-loading proxy** mexanizmini seçirik. **Proxy** EF-in Order sinfindən törədib yaratdığı xüsusi obyekt variantıdır. Törəmə sinif başqa sinfin imkanlarını götürüb genişləndirən sinifdir.

Terminal - layihəyə uyğun paket əlavə edilir

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies --version 10.0.9
```

İşlək layihədə paket artıq var. **Paket** layihəyə əlavə edilən hazır kitabxanadır. Context ayarını quran builder üzərində bunu aktivləşdiririk:

C# - layihənin ayarı

```
builder.UseLazyLoadingProxies();
```

C# - Order sinfindəki navigation

```
public virtual ICollection<OrderItem> Items { get; set; }
    = new List<OrderItem>();
```

Getter xassə oxunanda işləyən hissədir. `virtual` törəmə sinfin həmin davranışı dəyişməsinə imkan verir. **Override** törəmə sinifdə həmin davranışın yenidən təyin edilməsidir. Proxy Items getter-inə müdaxilə edib əlaqənin yüklənməsini başladır, sonra kolleksiyanı qaytarır.

Bu proxy variantında Order sinfi törədilə bilən olmalıdır; nümunədə public-dır və sealed deyil. **Sealed** sinifdən törəmə yaratmağı qadağan edir. **Tək virtual yazmaq kifayət deyil:** paket, ayar və EF-in yaratdığı uyğun proxy obyekt də lazımdır.

EF-də proxy-siz lazy mexanizmi də mövcuddur. Buradakı virtual tələbi məhz seçdiyimiz proxy üsuluna aiddir.

new Order ilə EF-in verdiyi Order eyni davranmır

C# - adi obyekt; bazaya sorğu yoxdur

```
var plain = new Order { Id = 42 };
Console.WriteLine(plain.GetType().Name); // Order
Console.WriteLine(plain.Items.Count);   // 0
```

Id-ni 42 yazmağımız bazadakı Order 42-ni gətirmədi. Sadəcə yeni C# obyektinə nömrə verdik. Onun Items-i sinifdə yaratdığımız boş List-dir. **Bu modeldə adi new Order() proxy yaratmır və təkcə Items-i oxumaq bazaya getmir.**

C# - proxy aktiv ayarlarla bazadan oxuyuruq

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
Console.WriteLine(order.GetType().Name); // OrderProxy
```

`GetType()` obyektin həqiqi işləmə vaxtı tipini göstərir. Bu sınaqda EF-in yaratdığı tipin adı OrderProxy-dir. Bu tipin adı kitabxana detalıdır; kodun işləməsini həmin ada bağlamırıq.

OrderProxy Order-dan törədiyi üçün onu Order dəyişənində saxlaya bilirik. Sən kodda `Order` görürsən, amma xassə oxunuşunda xüsusi davranışa malik obyektlə işləyirsən.

Məntiqi yol - EF-in mənbə kodu deyil

```
order.Items oxunur
-> proxy yüklənməmiş əlaqəni aşkar edir
-> EF OrderId = 42 məhsullarını oxuyur
-> obyektlər Items-ə yerləşdirilir
-> getter hazır kolleksiyanı qaytarır
```

Paket əlavə etmək əvvəldən yaratdığın bütün adi Order obyektlərini öz-özünə proxy-yə çevirmir.

Items və Count-u iki ayrı addımda gör

C# - lazy aktivdir, context təzədir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; Items hələ yüklənməyib.

var items = order.Items;
// Getter məhsulları gətirdi; SQL: 2.

var count = items.Count;
// Hazır kolleksiyanın sayıdır: 2; SQL yenə 2.

var again = order.Items;
// Əlaqə artıq yüklənib; SQL yenə 2.
```

`order.Items.Count` bu nümunədə əvvəl `order.Items`-i oxuyur, sonra alınmış kolleksiyanın Count-unu oxuyur. **SQL COUNT əmri işləmir. Məhsul obyektləri gətirilir, sonra onların sayı alınır.**

SQL - ikinci əmrin sadələşdirilmiş mənası

```
SELECT Id, OrderId, ProductName
FROM OrderItems
WHERE OrderId = 42;
```

İlk getter oxunuşu bitəndə kolleksiyada Cay və Tort obyektləri var, əlaqə yüklənmiş sayılır. Növbəti adi oxunuş həmin kolleksiyanı qaytarır; avtomatik olaraq hər dəfə bazanı yeniləmir.

Bu davranış yüklənməmiş əlaqə üçündür. Əgər əvvəl Include və ya LoadAsync ilə əlaqəni gətirmiş olsaydıq, getter bu təcrübədə yenidən sorğu işlətməzdi.

Context istifadəyə açıq qalmalıdır; bağlanmış context ilə hələ yüklənməmiş əlaqəni gətirməyə arxalanma. Əsas sorğudakı await getter-i asinxron etmir: bu proxy getter-i sinxron yükləyir. Asinxron yükləmə anını özün göstərmək üçün explicit LoadAsync istifadə edə bilərsən.

İşlədərək gör: hansı addım SQL sayını artırır?

İşlək layihə qovluğunda `dotnet run -- lazy-steps` əmrini ver. Əvvəl adi new obyekt yoxlanır, sonra təzə context-dən Order 42 alınır. Çıxışın əsas hissəsi belədir:

Yoxlanmış nəticə

```
New tip: Order
New Items: 0
New SQL: 0
EF tip: OrderProxy
1) SQL: 1
1) Yuklenib: False
2) Ad: Aysel
2) SQL: 1
3) SQL: 2
3) Yuklenib: True
4) Items: 2
4) SQL: 2
5) Eyni kolleksiya: True
5) SQL: 2
```

1: əsas Order alınıb; Items yüklənməyib. Yüklənmə vəziyyətini Items-ə toxunmadan `db.Entry(order).Collection(o => o.Items).IsLoaded` ilə yoxlayırıq.

2: CustomerName oxunub; bu dəyər artıq vardı, əlavə SQL yoxdur.

3: `var items = order.Items;` yazılıb. Təkcə bu getter oxunuşu ikinci SQL-i başladıb; əlaqə artıq yüklənib.

4: `items.Count` 2-ni qaytarıb; əlavə SQL yoxdur.

5: Items yenidən oxunub. `ReferenceEquals` iki istinadın eyni obyektə gedib-getmədiyini yoxlayır. Burada eyni kolleksiya qaytarılıb və SQL sayı artmayıb. Çıxışın ardınca faktiki iki SQL qeydi də göstərilir.

Explicit və lazy: eyni nəticə, fərqli başlanğıc

Hər iki üsulda əvvəl Order, sonra onun Items məlumatı gətirilə bilər. Əsas fərq ikinci sorğunu **hansı əməliyyatın başlatmasıdır**. Bu iki nümunə ayrı təzə context-lərdə işlədilir.

C# - explicit; lazy loading bağlıdır

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1

await db.Entry(order).Collection(o => o.Items).LoadAsync();
// İkinci SQL məhz bu açıq əmrdə işləyir.

var count = order.Items.Count; // 2; SQL cəmi 2-dir.
```

LoadAsync “indi bu order-in məhsullarını yüklə” əmridir. Əmri hansı şərtdə və hansı anda çağıracağını özün seçirsən. Lazy bağlı bu təcrübədə həmin əmri yazmadan Items.Count oxumaq məhsulları gətirməz.

C# - lazy aktivdir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1

var count = order.Items.Count;
// Items getter-i ikinci SQL-i işlədir, sonra Count 2 olur.
```

İkincidə ayrıca yükləmə əmri yazmadıq. Uyğun proxy-nin Items getter-i həmin işi başladı. **Explicit-də yükləmə çağırışı kodda açıqdır; lazy-də navigation oxunuşunun içindədir.** Bu nümunələrdə hər ikisi cəmi iki SQL işlədir. Fərq təkcə SQL sayı deyil, yükləmə anını necə idarə etməyimizdir.

Hansı davranışı əvvəlcədən görə bilirsən?

Sual	Explicit	Lazy proxy üsulu
Əlavə SQL-i başladan nədir?	Yazdığın LoadAsync çağırışı.	Yüklənməmiş Items getter-inin oxunuşu.
Yükləmə əmrini keçsən nə olur?	Lazy bağlı bu nümunədə məhsullar gətirilmir; Items boş qalır.	Items oxunmasa məhsul sorğusu yoxdur; oxunsa işləyir.
Proxy və virtual lazımdır?	Bu LoadAsync nümunəsi üçün yox.	Bu seçilmiş proxy variantı üçün lazımdır.
Asinxron yükləmə necədir?	LoadAsync-i await edirsən.	Adi getter bu nümunədə sinxron yükləyir.
Təkrar adi oxunuş necədir?	Artıq gətirilmiş kolleksiya oxunur.	Əlaqə yüklənmişdirsə getter yeni SQL göndərmir.

Məsələn, istifadəçinin məhsul təfərrüatını görmək icazəsini yoxlayıb sonra məlumatı gətirmək istəyirsən. Explicit-də yoxlamadan sonra LoadAsync yaza bilərsən. Yükləmənin yeri görünür.

Lazy-də isə məhsullara sadə müraciət də baza işi ola bilər. Sifarişləri göstərən dövrün içində `order.Items.Count` yazmaq hər sifariş üçün əlavə SQL yarada bilər.

“Explicit işlətsəm əlavə sorğu problemi həll olunur” nəticəsini çıxarma. Hər sifariş üçün ayrıca LoadAsync yazsan, aşağıdakı N+1 nümunəsində eyni sayda SQL işləyəcək. SQL-in açıq görünməsi onun avtomatik olaraq az olması demək deyil.

N+1-də N nəyi sayır?

Bu nümunədə **N əsas sorğudan gələn sifarişlərin sayıdır**. Əvvəl bir SQL ilə N sifariş alınır, sonra hər sifarişin məhsulları üçün ayrıca bir SQL işləyir. Yekun: **1 + N SQL**.

Bazada üç sifarişimiz var: 42-nin iki məhsulu, 43-ün bir məhsulu, 44-ün isə heç məhsulu yoxdur. Lazy aktivdir, əlaqələr əvvəl yüklənməyib:

C# - nplus1-lazy rejiminin məntiqi

```
var orders = await db.Orders.OrderBy(o => o.Id)
    .ToListAsync();
// 3 sifariş alındı. SQL: 1.

foreach (var order in orders)
{
    var count = order.Items.Count;
    Console.WriteLine(count);
}
```

Addım	Nə gətirilir?	Cəmi SQL
ToListAsync	42, 43, 44 sifarişləri	1
42.Items	Cay və Tort	2
43.Items	Qehve	3
44.Items	Uyğun məhsul tapılmır	4

44-də məhsul olmadığı üçün onun sorğusu ləğv olunmur. Məhsulun olmadığını müəyyən etmək üçün də bazadan soruşulur. N = 3 olduğuna görə cəmi **1 + 3 = 4 SQL** işləyir. Məhsul obyektlərinin ümumi sayı ilə N-i qarışdırma; burada əsas siyahıdakı Order-ları sayırıq.

Explicit dövrü də N+1 yarada bilər

Lazy-ni bağlayaq və hər sifariş üçün yükləmə əmrini özümüz yazaq:

C# - nplus1-explicit rejiminin məntiqi

```
var orders = await db.Orders.OrderBy(o => o.Id)
    .ToListAsync();
// SQL: 1

foreach (var order in orders)
{
    await db.Entry(order).Collection(o => o.Items)
        .LoadAsync();
    Console.WriteLine(order.Items.Count);
}
// SQL: 4
```

42 üçün bir LoadAsync, 43 üçün bir LoadAsync, 44 üçün bir LoadAsync işləyir. Əsas sorğu ilə birlikdə yenə dörd SQL-dir. **Lazy-də getter-in içində olan əlavə sorğuları bu dəfə açıq əmrlərlə yazdıq; say dəyişmədi.**

Niyə bu problem ola bilər? Hər əlavə əmr baza ilə ayrıca əlaqə işi və gözləmə yaradır. Eyni quruluş yüz sifarişdə hər biri üçün ayrıca oxuma edərsə 101 SQL işləyə bilər. Gecikmə və baza yükü artar. Dəqiq vaxt məlumatın ölçüsündən, bazadan və bağlantıdan asılıdır.

`await` yazmaq bu əmrləri bir SQL-də birləşdirmir; hər yükləməni ayrıca gözləyirik. N+1 icra quruluşudur, yalnız lazy loading-in başqa adı deyil.

Əsas şərtləri də unutma: əlaqələr əvvəl gətirilməyib və hər sifarişin əlaqəsini oxuyuruq. Əvvəldən yüklənmiş əlaqələr və yalnız bir neçə sifarişə müraciət faktiki SQL sayını dəyişə bilər.

Hamısının məhsulları lazımdırsa əvvəlcədən istə

Bu ekranda bütün sifarişlərin məhsullarını göstərəcəyimizi bilirik. Ona görə hər sifarişdə ayrıca yükləmək əvəzinə Include ilə tələbi əvvəl yazırıq:

C# - nplus1-eager rejiminin məntiqi

```
var orders = await db.Orders.Include(o => o.Items)
    .OrderBy(o => o.Id).ToListAsync();
// Bu model və ayarlarda SQL: 1.

foreach (var order in orders)
    Console.WriteLine(order.Items.Count);
// 2, 1, 0; SQL cəmi yenə 1-dir.
```

Artıq Items obyektləri gəlmişdir. Dövr onları oxuyur; hər Order üçün yeni məhsul SQL-i işləmir. Bu, buradakı N+1 quruluşunu aradan qaldırır.

Eager istifadə olunan hər layihədə mütləq bir SQL olacağını demirik. Əsas məqsəd **hər element üçün ayrıca sorğu yaranıb-yaranmadığını** yoxlamaqdır. Bu nümunədə bir SQL olduğunu faktiki icra qeydlərində ölçmüşük.

İşlək layihədə üç rejimi ayrı-ayrı işlət:

Terminal

```
dotnet run -- nplus1-lazy
dotnet run -- nplus1-explicit
dotnet run -- nplus1-eager
```

İlk iki rejimdə `Orders SQL: 1`, sonra sifarişlər üzrə SQL 2, 3, 4 görünür. Eager rejimində bütün addımlarda SQL 1 qalır. Hər üçündə məhsul sayları **2, 1, 0**-dır. Ardınca real SQL qeydləri yazılır; eyni nəticəni neçə ayrı əmrlə aldığını görə bilirsən.

Üç variantı bir zaman xəttində müqayisə et

Hər sınaqda eyni 42 nömrəli sifarişi alırıq. Ayarlar əvvəlki səhifələrdəki kimidir və başlanğıcda əlaqələr yüklənməyib.

Addım	Eager	Explicit	Lazy
Sorğunu qururuq	Include var; SQL 0	Include yox; SQL 0	Include yox; SQL 0
SingleAsync bitir	Order + Items hazır; SQL 1	Order hazır; SQL 1	Order hazır; SQL 1
Sonrakı addım	Items.Count	LoadAsync, sonra Count	Items.Count
Əlavə SQL-i başladan	Əlavə SQL yoxdur	LoadAsync	İlk Items getter-i
Yekun Items / SQL	2 / 1	2 / 2	2 / 2

Explicit və lazy-nin burada SQL sayı eynidir, amma nəzarət nöqtəsi fərqlidir. Explicit-də yükləmə əmrini kodda görürsən. Lazy-də uyğun xassə oxunuşu bu işi başladır.

Eager və deferred bir yerdə ola bilər. Include yazıb sonra ToListAsync gözləmək bunun nümunəsidir. Eager əlaqənin tələb olunma qaydasıdır, deferred əsas sorğunun nəticəsinin nə vaxt oxunmasıdır.

IEnumerable və lazy də eyni anlayış deyil. IEnumerable elementləri oxumaq müqaviləsidir. Lazy loading isə entity əlaqəsinə müraciətdə ayrıca məlumat gətirmək mexanizmidir.

Özün yoxla: `var q = db.Orders.Include(o => o.Items);` sətrində SQL sayı neçədir? **0.** Items-in lazım olduğunu dedik, amma nəticəni hələ istəmədik.

Bu dörd mövzunu ayırd etmək üçün

Deferred execution: “Əsas sorğunu nə vaxt işlədirəm?” Sorğunu hazırlayıb nəticəsini ToList, foreach və ya başqa nəticə istəyən əməliyyata qədər gözlətməkdir.

IQueryable: “Sorğuya hansı şərtləri əlavə edə bilirəm?” EF provider-i uyğun ifadələri SQL-ə çevirə bilir. Təkcə IQueryable adı mənbənin mütləq SQL bazası olduğunu bildirmir.

IEnumerable: “Elementləri necə oxuyuram?” Bu müqavilə hazır List-i də, hələ işlənməmiş mənbəni də ifadə edə bilər. Məlumatın artıq gəldiyini yalnız tipin adından bilmək olmur.

Loading: “Əlaqəli Items məlumatını hansı anda gətirirəm?” Eager-də sorğuda Include, explicit-də ayrıca LoadAsync, aktiv lazy-də isə yüklənməmiş navigation-un ilk oxunuşu bunu müəyyən edir.

N+1: “Hər əsas element üçün ayrıca əlaqə sorğusu göndərirəmmi?” Əvvəl bir SQL ilə N sifariş alıb, sonra hər birinin məhsullarını bir ayrıca SQL ilə istəsək cəmi 1 + N əmri alınır. Bu, həm lazy, həm explicit dövründə yarana bilər.

Hər kodda əvvəl mənbəni və görünən tipi müəyyən et. Sonra SQL-i başladan sətri tap. Axırda Order-un və Items-in hansı anda hazır olduğunu de.

Rəsmi istinadlar:

- IQueryable, AsEnumerable və deferred execution.
- Eager loading, explicit loading və lazy loading.
- Əlavə əlaqə sorğuları və N+1.